

1. Create one IAM user and assign EC2 and S3 full access roles.

Steps I Used

1. Opened AWS Management Console

- Logged in to the AWS Management Console.
- Navigated to IAM (Identity and Access Management).

2. Created an IAM user

- Went to IAM → Users → Create user.
- Created the required IAM user.
- Completed the user creation process.

3. Granted EC2 Full Access

- Opened the newly created IAM user.
- Went to Permissions → Add permissions.
- Selected Attach policies directly.
- Searched for and selected:

AmazonEC2FullAccess

4. Granted S3 Full Access

- Also selected:

AmazonS3FullAccess

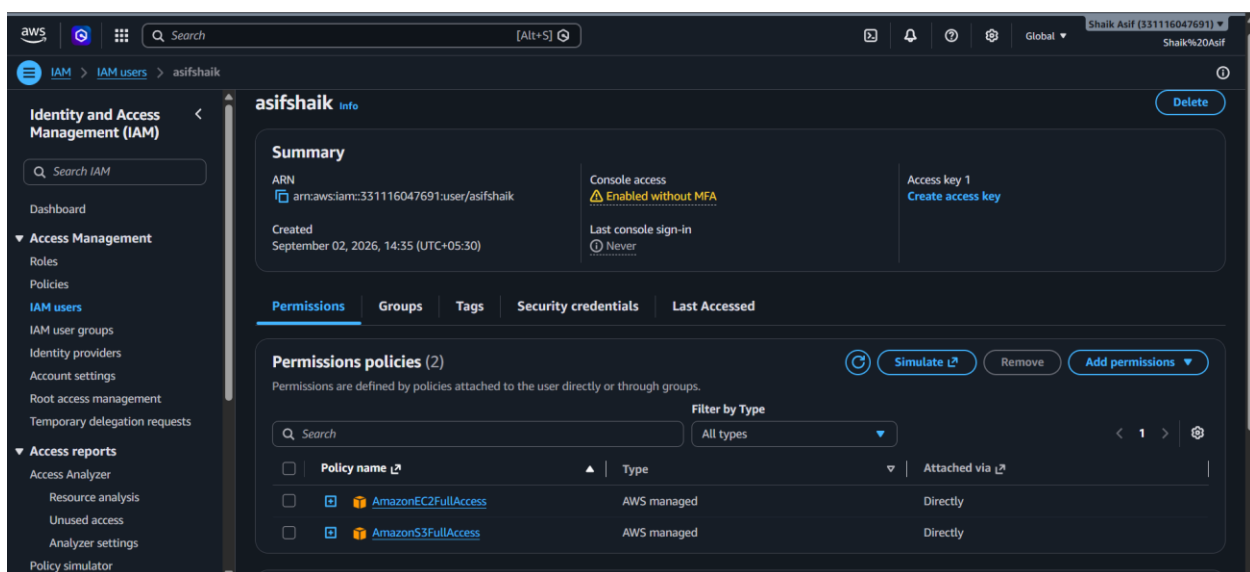
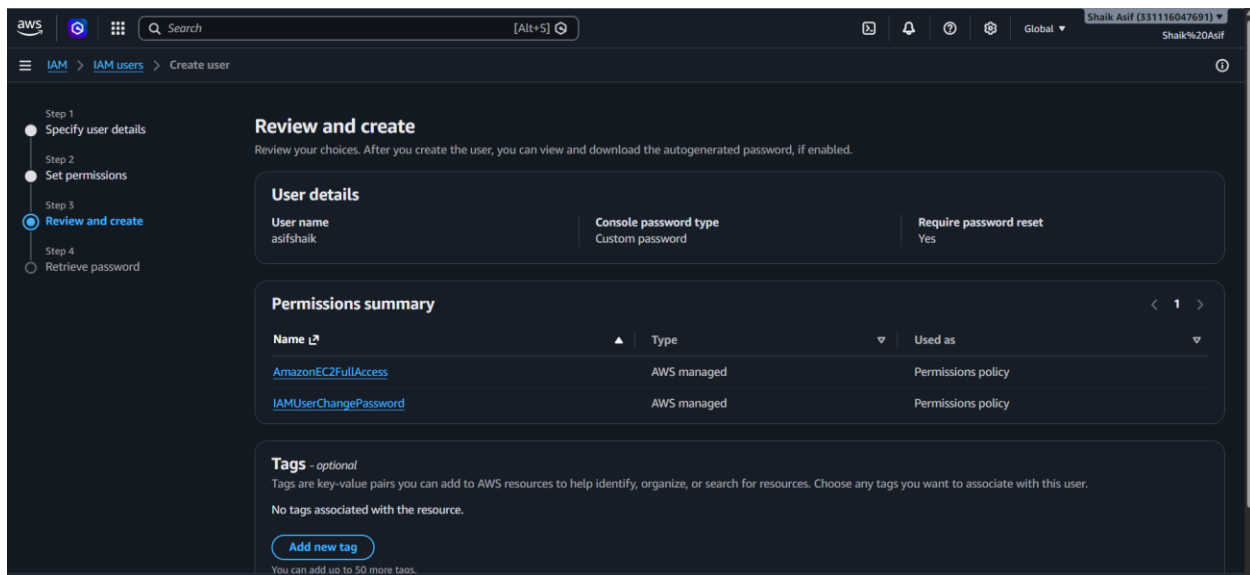
- This gives the IAM user full permissions to work with EC2 and S3 resources.

5. Reviewed the permissions

- Verified that both policies were attached to the IAM user:
 - AmazonEC2FullAccess
 - AmazonS3FullAccess

Permissions Assigned

Policy	Purpose
AmazonEC2FullAccess	Provides full access to Amazon EC2 resources
AmazonS3FullAccess	Provides full access to Amazon S3 resources



Conclusion

An IAM user was successfully created and the required permissions were assigned. The user was granted AmazonEC2FullAccess and AmazonS3FullAccess, allowing the user to manage EC2 and S3 resources. Thus, Task 1 was completed successfully.

2. Create one group in IAM and assign read access for EC2.

Steps I Used

1. Opened AWS Management Console

- Logged in to the AWS Management Console.
- Navigated to IAM (Identity and Access Management).

2. Created an IAM Group

- Went to IAM → User groups.
- Clicked Create group.
- Created the group named:

EC2-ReadOnly

3. Selected EC2 Read-Only Policy

- While creating the group, went to Permissions policies.
- Searched for:

AmazonEC2ReadOnlyAccess

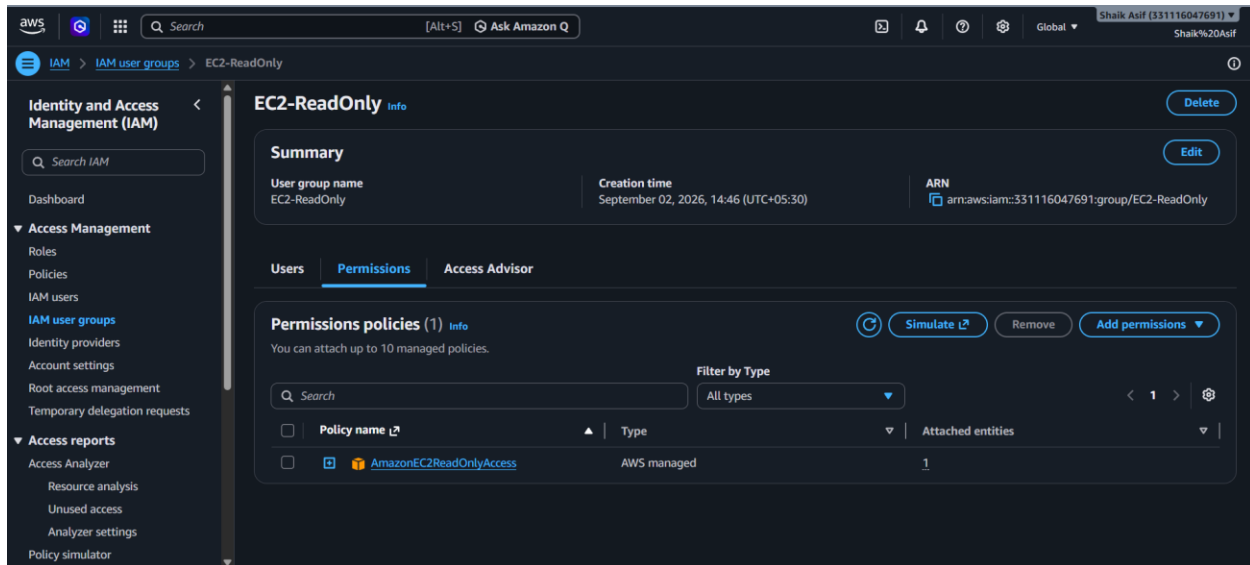
- Selected the policy and attached it to the group.

4. Verified the Group Permissions

- Opened the EC2-ReadOnly group.
- Checked the Permissions tab.
- Verified that AmazonEC2ReadOnlyAccess was attached.

Permission Assigned

Group	Policy	Purpose
EC2-ReadOnly	AmazonEC2ReadOnlyAccess	Allows users to view EC2 resources but does not allow them to create, modify, or terminate EC2 resources.



Conclusion

An IAM group named EC2-ReadOnly was successfully created and the AmazonEC2ReadOnlyAccess policy was attached to it. This group provides read-only access to EC2 resources, allowing members to view EC2 information without having permission to make changes. Thus, Task 2 was completed successfully.

3. Create a new user named "Devops" and add to the group created in task 2.

Steps I Used

1. Opened AWS Management Console

- Logged in to the AWS Management Console.
- Navigated to IAM → Users.

2. Created the IAM User

- Clicked Create user.
- Created the user named:

Devops

3. Added the User to the IAM Group

- During user creation, selected Add user to group.
- Selected the existing group:

EC2-ReadOnly

- Completed the user creation.

4. Verified Group Membership

- Opened IAM → Users → Devops.
- Checked the Groups section.
- Verified that Devops was a member of:

EC2-ReadOnly

5. Verified Inherited Permissions

- Checked the permissions available to the Devops user.
- Since the EC2-ReadOnly group has the policy:

AmazonEC2ReadOnlyAccess

- The Devops user inherits the same EC2 read-only permissions through the group.

Permission Flow

IAM Group: EC2-ReadOnly



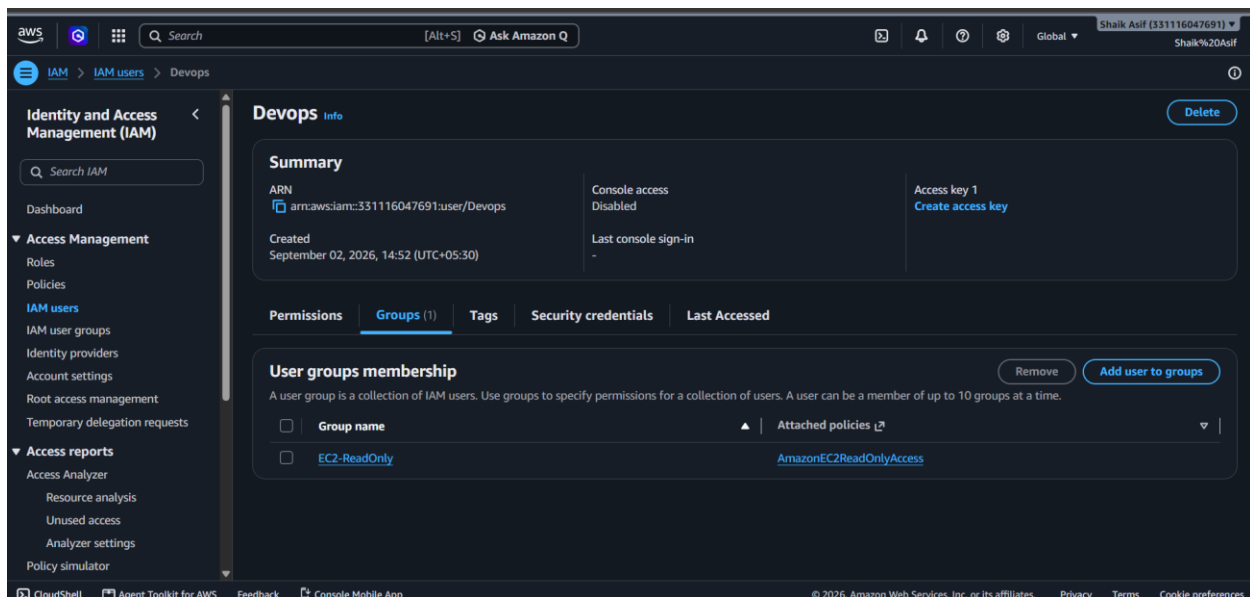
AmazonEC2ReadOnlyAccess



IAM User: Devops



EC2 Read-Only Access



Conclusion

The IAM user Devops was successfully created and added to the existing EC2-ReadOnly group. The user inherited the AmazonEC2ReadOnlyAccess policy from the group, allowing the user to view EC2 resources without modifying them. Thus, Task 3 was completed successfully.

4. Write a bash script to create an IAM user with VPC full access.

Steps I Used

1. Opened Git Bash

- I used Git Bash to execute the Bash script and AWS CLI commands.

2. Created a Bash script

- Created a script to automate IAM user creation and policy attachment.
- The user name used was:

vpc-admin

3. Added IAM user creation command

```
aws iam create-user --user-name "$USER_NAME"
```

This command creates the IAM user if the user does not already exist.

4. Added VPC Full Access policy

I used the AWS managed policy:

AmazonVPCFullAccess

with the policy ARN:

```
arn:aws:iam::aws:policy/AmazonVPCFullAccess
```

The script attached it using:

```
aws iam attach-user-policy \  
  --user-name "$USER_NAME" \  
  --policy-arn "$POLICY_ARN"
```

5. Added error handling

- I used \$? to check whether each AWS CLI command completed successfully.
- If a command failed, the script displayed an error message and exited.

6. Handled the existing user

- When I ran the script, vpc-admin already existed.
- I therefore used an idempotent version of the script that first checks whether the user exists before attempting to create it.

The check was:

```
aws iam get-user --user-name "$USER_NAME"
```

If the user already existed, the script continued with the policy attachment.

7. Verified the IAM user in AWS Console

- Navigated to IAM → Users → vpc-admin.
- Checked the Permissions section.
- Verified that:

AmazonVPCFullAccess was attached to vpc-admin.

Result

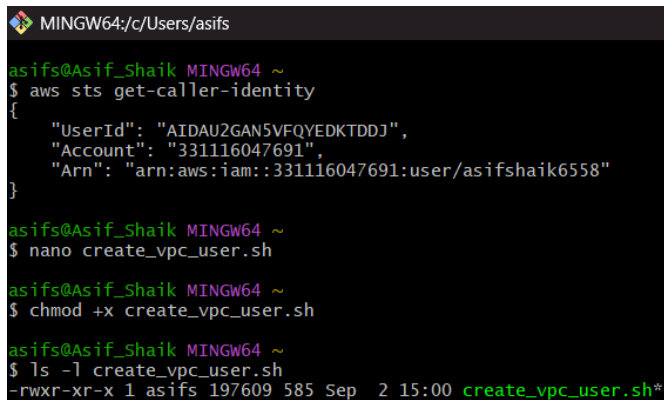
IAM User: vpc-admin

↓

AmazonVPCFullAccess

↓

Full access to Amazon VPC resources



```
MINGW64/c/Users/asifs
asifs@Asif_Shaik MINGW64 ~
$ aws sts get-caller-identity
{
  "UserId": "AIDAU2GAN5VFQYEDKTDJ",
  "Account": "331116047691",
  "Arn": "arn:aws:iam::331116047691:user/asifshaik6558"
}

asifs@Asif_Shaik MINGW64 ~
$ nano create_vpc_user.sh

asifs@Asif_Shaik MINGW64 ~
$ chmod +x create_vpc_user.sh

asifs@Asif_Shaik MINGW64 ~
$ ls -l create_vpc_user.sh
-rwxr-xr-x 1 asifs 197609 585 Sep  2 15:00 create_vpc_user.sh*
```

```
MINGW64:/c/Users/asifs
GNU nano 6.3 create_vp
#!/bin/bash

USER_NAME="vpc-admin"

echo "Creating IAM user: $USER_NAME"

aws iam create-user --user-name "$USER_NAME"

if [ $? -eq 0 ]; then
    echo "User $USER_NAME created successfully."
else
    echo "Failed to create user."
    exit 1
fi

echo "Attaching VPC Full Access policy..."

aws iam attach-user-policy \
    --user-name "$USER_NAME" \
    --policy-arn arn:aws:iam::aws:policy/AmazonVPCFullAccess

if [ $? -eq 0 ]; then
    echo "AmazonVPCFullAccess attached successfully."
else
    echo "Failed to attach VPC policy."
    exit 1
fi

echo "Task 4 completed successfully."
```

```
MINGW64:/c/Users/asifs
asifs@Asif_Shaik MINGW64 ~
$ aws iam get-user
{
  "User": {
    "Path": "/",
    "UserName": "asifshaik6558",
    "UserId": "AIDAU2GAN5VFQYEDKTDDJ",
    "Arn": "arn:aws:iam::331116047691:user/asifshaik6558",
    "CreateDate": "2026-08-25T17:08:01+00:00",
    "Tags": [
      {
        "Key": "AKIAU2GAN5VFM3PUIV2",
        "Value": "asifshaik6558"
      }
    ]
  }
}

asifs@Asif_Shaik MINGW64 ~
$ aws iam list-attached-user-policies --user-name asifshaik6558
{
  "AttachedPolicies": [
    {
      "PolicyName": "AmazonEC2FullAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/AmazonEC2FullAccess"
    },
    {
      "PolicyName": "IAMFullAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/IAMFullAccess"
    }
  ]
}

asifs@Asif_Shaik MINGW64 ~
$ ./create_vpc_user.sh
Creating IAM user: vpc-admin
{
  "User": {
    "Path": "/",
    "UserName": "vpc-admin",
    "UserId": "AIDAU2GAN5VFS33NMUWU7",
    "Arn": "arn:aws:iam::331116047691:user/vpc-admin",
    "CreateDate": "2026-09-02T09:37:54+00:00"
  }
}
User vpc-admin created successfully.
Attaching VPC Full Access policy...
AmazonVPCFullAccess attached successfully.
Task 4 completed successfully.
```

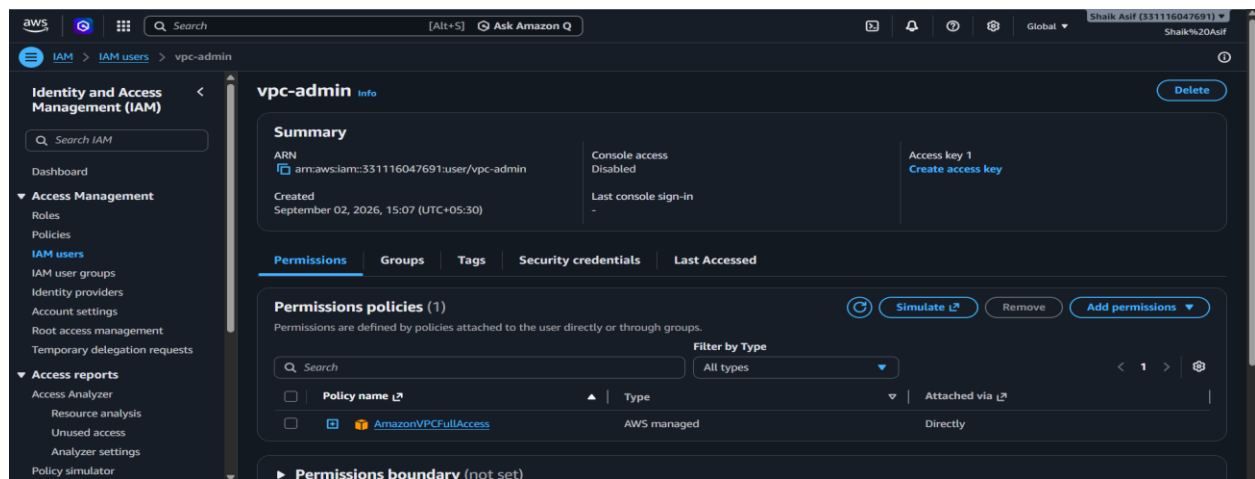
```

asifs@Asif_Shaik MINGW64 ~
$ ./create_vpc_user.sh
Creating IAM user: vpc-admin
{
  "User": {
    "Path": "/",
    "UserName": "vpc-admin",
    "UserId": "AIDAU2GAN5VFS33NMUWU7",
    "Arn": "arn:aws:iam::331116047691:user/vpc-admin",
    "CreateDate": "2026-09-02T09:37:54+00:00"
  }
}
User vpc-admin created successfully.
Attaching VPC Full Access policy...
AmazonVPCFullAccess attached successfully.
Task 4 completed successfully.

asifs@Asif_Shaik MINGW64 ~
$ aws iam get-user --user-name vpc-admin
{
  "User": {
    "Path": "/",
    "UserName": "vpc-admin",
    "UserId": "AIDAU2GAN5VFS33NMUWU7",
    "Arn": "arn:aws:iam::331116047691:user/vpc-admin",
    "CreateDate": "2026-09-02T09:37:54+00:00"
  }
}

asifs@Asif_Shaik MINGW64 ~
$ aws iam list-attached-user-policies --user-name vpc-admin
{
  "AttachedPolicies": [
    {
      "PolicyName": "AmazonVPCFullAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/AmazonVPCFullAccess"
    }
  ]
}

```



Conclusion

An IAM user named vpc-admin was created/configured using a Bash script, and the AmazonVPCFullAccess AWS managed policy was attached to the user. The IAM configuration was verified through the AWS Console, confirming that vpc-admin has full access to Amazon VPC resources. Thus, Task 4 was completed successfully.

5. Create an IAM policy to allow EC2 access for a specific user in specific regions only.

Steps I Used

1. Opened AWS Management Console

- Logged in to the AWS Management Console.
- Navigated to IAM → Policies.

2. Created a custom IAM policy

- Clicked Create policy.
- Selected the JSON policy editor.
- Created a policy named:

EC2-Specific-Regions

3. Defined the policy

I used the following policy to allow EC2 access only in Mumbai (ap-south-1) and N. Virginia (us-east-1):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowEC2OnlyInSpecificRegions",
      "Effect": "Allow",
      "Action": "ec2:*",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "aws:RequestedRegion": [
            "ap-south-1",
            "us-east-1"
          ]
        }
      }
    }
  ]
}
```

4. Created a dedicated IAM user

- Created the IAM user:

EC2-Region-User

- Attached the custom policy EC2-Specific-Regions to this user.

5. Configured AWS CLI profile

- Created a separate CLI profile for the user:

`aws configure --profile ec2-region`

- Configured the access key and secret key for EC2-Region-User.
- Set the default region to ap-south-1.

6. Verified the IAM user

I ran:

`aws sts get-caller-identity --profile ec2-region`

The output confirmed that the CLI was using:

`arn:aws:iam::331116047691:user/EC2-Region-User`

7. Tested EC2 access in an allowed region

I tested Mumbai:

`aws ec2 describe-instances \`
`--region ap-south-1 \`
`--profile ec2-region`

The command executed successfully because ap-south-1 was an allowed region.

8. Tested EC2 access in a restricted region

I tested Ireland:

`aws ec2 describe-instances \`
`--region eu-west-1 \`
`--profile ec2-region`

The command returned UnauthorizedOperation, proving that EC2 access was restricted outside the specified regions.

Result

EC2-Region-User



EC2-Specific-Regions Policy



Allowed
ap-south-1
us-east-1

eu-west-1 → Access Denied

The screenshot shows the AWS IAM console interface for creating a new policy. The breadcrumb navigation at the top reads 'IAM > Policies > Create policy'. On the left, a sidebar indicates 'Step 1: Specify permissions' is the current step, with 'Step 2: Review and create' listed below it. The main heading is 'Specify permissions' with an 'Info' icon. Below the heading is a descriptive text: 'Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.' The 'Policy editor' section is active, showing a JSON configuration for a policy statement. The JSON is as follows:

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Sid": "AllowEC2OnlyInSpecificRegions",
6       "Effect": "Allow",
7       "Action": "ec2:*",
8       "Resource": "*",
9       "Condition": {
10        "StringEquals": {
11          "aws:RequestedRegion": [
12            "ap-south-1",
13            "us-east-1"
14          ]
15        }
16      }
17    ]
18  }
```

On the right side of the JSON editor, there are tabs for 'Visual', 'JSON' (which is selected), and 'Actions'. Below the tabs is an 'Edit statement' section. It contains the text 'Select a statement' and 'Select an existing statement in the policy or add a new statement.' At the bottom of this section is a button labeled '+ Add new statement'.

```
MINGW64/c/Users/asifs
asifs@Asif-Shaik MINGW64 ~
$ aws configure --profile ec2-region

Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the command 'aws login'.

AWS Access Key ID [None]: AKIAU2GAN5VF5B20HDSY
AWS Secret Access Key [None]: 6ACcER86bc33+CrWjyZTcyoS/ImWldLTnVhpn9tH
Default region name [None]: ap-south-1
Default output format [None]: json

asifs@Asif-Shaik MINGW64 ~
$ aws sts get-caller-identity --profile ec2-region
{
  "UserId": "AIDAU2GAN5VF5B20HDSY",
  "Account": "331116047691",
  "Arn": "arn:aws:iam::331116047691:user/EC2-Region-User"
}

asifs@Asif-Shaik MINGW64 ~
$ aws ec2 describe-instances --region ap-south-1 --profile ec2-region
{
  "Reservations": [
    {
      "ReservationId": "r-0959ac44319e83804",
      "OwnerId": "331116047691",
      "Groups": [],
      "Instances": [
        {
          "Architecture": "x86_64",
          "BlockDeviceMappings": [
            {
              "DeviceName": "/dev/sda1",
              "Ebs": {
                "AttachTime": "2026-08-10T09:38:28+00:00",
                "DeleteOnTermination": true,
                "Status": "attached",
                "VolumeId": "vol-0ce53af26df9a1083",
                "EbscardIndex": 0
              }
            }
          ],
          "ClientToken": "bc3b0b30-b22a-44ac-9e5b-15142e789ae7",
          "EbsOptimized": true,
          "EnaSupport": true,
          "Hypervisor": "xen",
          "NetworkInterfaces": [
            {
              "Attachment": {
                "AttachTime": "2026-08-10T09:38:27+00:00",
                "AttachmentId": "eni-attach-05739a19da7a0aa8f",
                "DeleteOnTermination": true,
                "DeviceIndex": 0,
                "Status": "attached",
                "NetworkCardIndex": 0
              },
              "Description": "",
              "Groups": []
            }
          ]
        }
      ]
    }
  ]
}
```

```
asifs@Asif-Shaik MINGW64 ~
$ aws ec2 describe-instances --region eu-west-1 --profile ec2-region

aws: [ERROR]: An error occurred (UnauthorizedOperation) when calling the DescribeInstances operation: You are not authorized to perform this operation. User: arn:aws:iam::331116047691:user/EC2-Region-User is not authorized to perform: ec2:DescribeInstances because no identity-based policy allows the ec2:DescribeInstances action
```

Conclusion

A custom IAM policy named EC2-Specific-Regions was successfully created and attached to the EC2-Region-User. The policy allows EC2 operations only in ap-south-1 and us-east-1. The configuration was validated using AWS CLI: EC2 access was successful in ap-south-1, while access to eu-west-1 was denied. Thus, Task 5 was completed successfully.

6. We have two accounts: Account A and Account B. Account A user should access an S3 bucket in Account B.

Steps I Used

1. Identified the two AWS accounts

- Account A: 331116047691 — my account
- Account B: 634176595009 — Karthik's account

2. Used the existing IAM user Asif in Account A

- I used the existing Asif IAM user instead of creating a new user.
- Created an access key for Asif so I could test the access using AWS CLI.

3. Created an S3 bucket in Account B

- Bucket name:

asif-karthik-cross-account-2026

- The bucket was created in Karthik's AWS account.

4. Added a bucket policy in Account B

The bucket policy allowed the Asif user from Account A:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowAsifAccess",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::331116047691:user/Asif"
      },
      "Action": [
        "s3:ListBucket",
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": [
        "arn:aws:s3:::asif-karthik-cross-account-2026",
        "arn:aws:s3:::asif-karthik-cross-account-2026/*"
      ]
    }
  ]
}
```

```
]
}
```

This allowed the Asif user from Account A to list the bucket and read/write objects.

5. Added an IAM policy to Asif in Account A

I created the inline policy CrossAccount-S3-Access:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowListBucket",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::asif-karthik-cross-account-2026"
    },
    {
      "Sid": "AllowObjects",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::asif-karthik-cross-account-2026/*"
    }
  ]
}
```

6. Configured a separate AWS CLI profile

I created the profile:

```
aws configure --profile asif-cross-account
```

I configured the credentials for the Asif user and selected ap-south-1 as the region.

7. Verified the AWS identity

```
aws sts get-caller-identity --profile asif-cross-account
```

Output confirmed:

Account: 331116047691

Arn: arn:aws:iam::331116047691:user/Asif

This confirmed that the CLI was using the correct Asif user.

8. Tested access to the bucket

```
aws s3 ls s3://asif-karthik-cross-account-2026 \  
--profile asif-cross-account \  
--region ap-south-1
```

The command completed without AccessDenied, confirming that Asif could access the bucket.

9. Created a test file

```
echo "Cross account S3 test" > test.txt
```

10. Uploaded the file to Account B's bucket

```
aws s3 cp test.txt s3://asif-karthik-cross-account-2026/ \  
--profile asif-cross-account \  
--region ap-south-1
```

Output:

```
upload: .\test.txt to s3://asif-karthik-cross-account-2026/test.txt
```

This successfully proved cross-account S3 access.

aws

Search

[Alt+S]

1

United States (N. Virginia)

CHVK (634176595009)

CHVK

Amazon S3

Buckets

asif-karthik-cross-account-2026

Successfully created bucket "asif-karthik-cross-account-2026"

asif-karthik-cross-account-2026

ObjectsMetadataPropertiesPermissionsMetricsManagementFile systems - newAccess Points

Objects (0)

Copy S3 URICopy URLDownloadOpenDeleteActionsCreate folderUpload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

NameTypeLast modifiedSizeStorage class

No objects

You don't have any objects in this bucket.

Upload

CloudShellAgent Toolkit for AWSFeedbackConsole Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. PrivacyTermsCookie preferences

aws

Search

[Alt+S]

1

United States (N. Virginia)

CHVK (634176595009)

CHVK

Amazon S3

Buckets

asif-karthik-cross-account-2026

Successfully edited bucket policy.

Public access is blocked because Block Public Access settings are turned on for this bucket

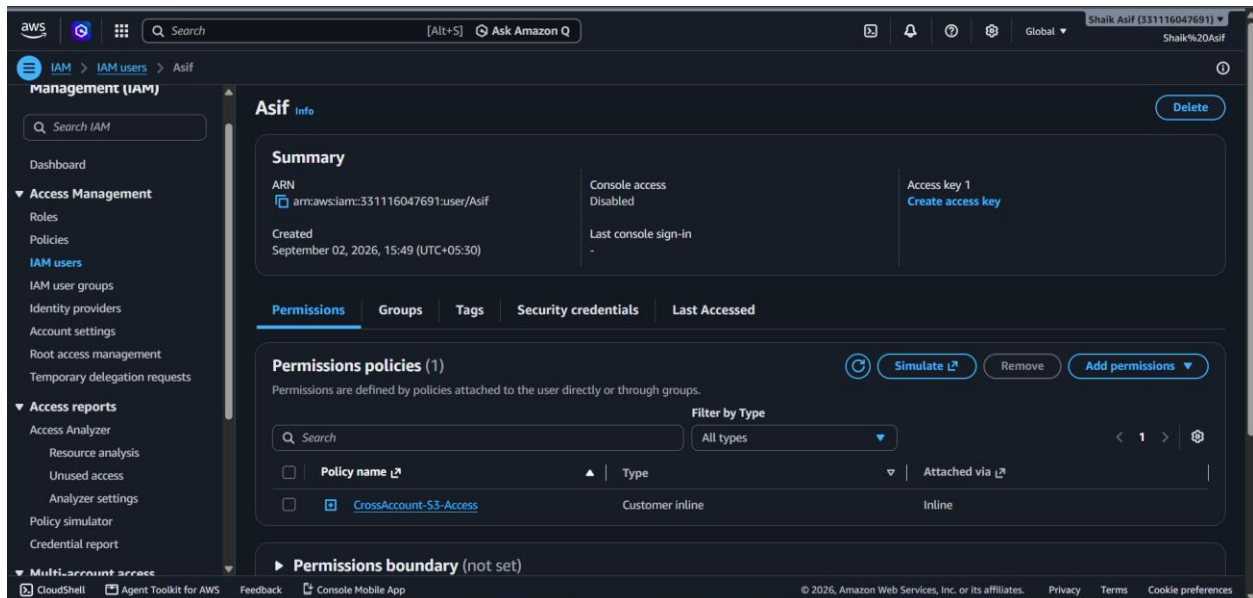
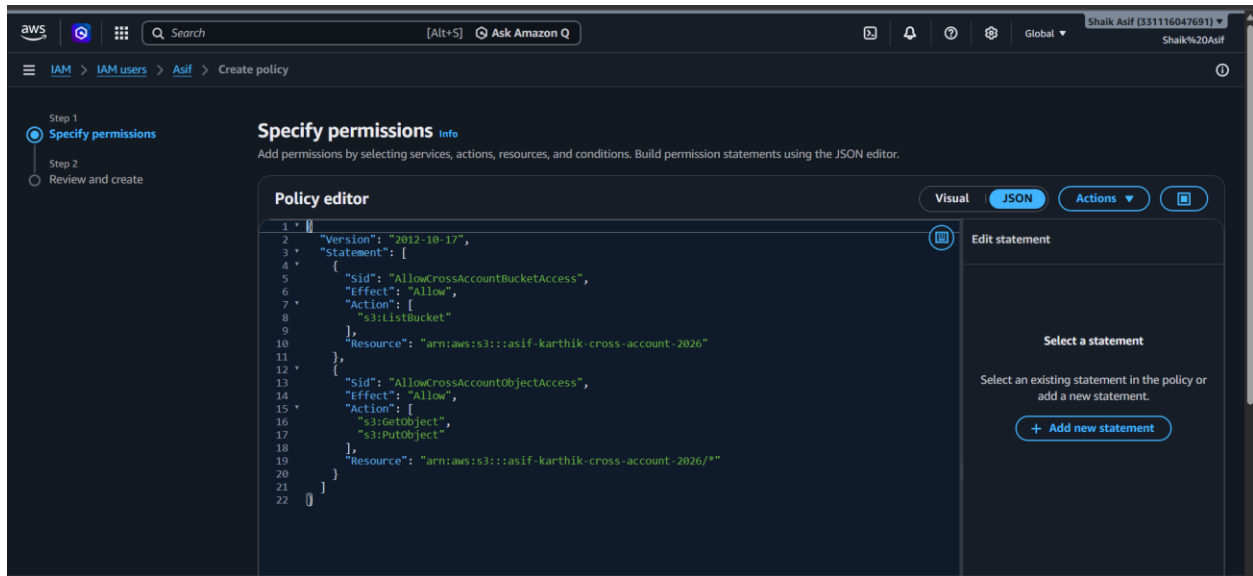
To determine which settings are turned on, check your Block Public Access settings for this bucket. [Learn more about using Amazon S3 Block Public Access](#)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowAsifAccess",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::331116047691:user/Asif"
      },
      "Action": [
        "s3:ListBucket",
        "s3:GetObject",
        "s3:PutObject"
      ]
    }
  ]
}
```

Copy

CloudShellAgent Toolkit for AWSFeedbackConsole Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. PrivacyTermsCookie preferences



```

asifs@Asif_Shaik MINGW64 ~
$ aws configure --profile asif-cross-account
AWS Access Key ID [*****HDSY]: AKIAU2GAN5VFVJ62WI6W
AWS Secret Access Key [*****n9tH]: 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
Default region name [ap-south-1]: ap-south-1
Default output format [json]: json

asifs@Asif_Shaik MINGW64 ~
$ aws sts get-caller-identity --profile asif-cross-account
{
  "UserId": "AIDAU2GAN5VFTUXQRRWUW",
  "Account": "331116047691",
  "Arn": "arn:aws:iam::331116047691:user/Asif"
}

asifs@Asif_Shaik MINGW64 ~
$ aws s3 ls s3://asif-karthik-cross-account-2026 --profile asif-cross-account --region ap-south-1

asifs@Asif_Shaik MINGW64 ~
$ aws s3 cp test.txt s3://asif-karthik-cross-account-2026/ --profile asif-cross-account --region ap-south-1
upload: .\test.txt to s3://asif-karthik-cross-account-2026/test.txt
  
```

Conclusion

Cross-account S3 access was successfully configured between two AWS accounts. IAM user Asif from Account A (331116047691) was granted permission to access the S3 bucket asif-karthik-cross-account-2026 located in Account B (634176595009). The configuration was successfully validated by identifying the correct IAM user, listing the bucket, and uploading test.txt from Account A to the S3 bucket in Account B. Therefore, Task 6 was completed successfully.